



IMPLEMENTAÇÃO DE ARRAYS SEM FUNÇÕES NATIVAS

2º SEMESTRE - ESTRUTURA DE DADOS

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



LIVRO DE REFERÊNCIA DA DISCIPLINA

- **BIBLIOGRAFIA BÁSICA:**

- GRONER, Loiane. **Estrutura de dados e algoritmos com JavaScript**: escreva um código JavaScript complexo e eficaz usando a mais recente ECMAScript. **São Paulo: Novatec Editora, 2019.**

- **NESTA AULA:**

- **Capítulo 3**



PARA SOBREVIVER AO JAVASCRIPT

Non-zero value



null



0



undefined



ARRAYS

*Exemplo de Implementação em JavaScript,
sem o uso de funções nativas*

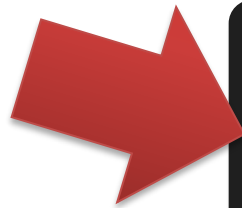
RELEMBRANDO VETORES...

- Vetores no Visualg (***também chamados de arrays ou arranjos***) são estruturas de dados que permitem armazenar vários valores do mesmo tipo em uma única variável, **acessados por um índice numérico**.
- A chave para entender vetores está no entendimento de que um valor guardado em vetor está em um determinado índice (posição)



RELEMBRANDO VETORES...

```
1 Algoritmo "Visualg_Para_Js"
2
3 Var
4   nomes : vetor[1..3] de caracter
5   i : inteiro
6
7 Inicio
8
9   nomes[1] <- "João"
10  nomes[2] <- "Maria"
11  nomes[3] <- "José"
12
13  Para i de 1 até 3 Faça
14    Escreval(nomes[i])
15  FimPara
16
17 Fimalgoritmo
```



```
1 // Declaração do vetor
2 let nomes = [];
3 nomes[0] = "João";
4 nomes[1] = "Maria";
5 nomes[2] = "José";
6 // Acessando cada posição do vetor
7 for (let i = 0; i < 3; i++) {
8   console.log(nomes[i]);
9 }
```

O QUE SÃO ARRAYS?

- Um array é uma estrutura de dados que armazena uma coleção de elementos (geralmente do mesmo tipo) em **posições consecutivas de memória**.
- Características principais:
 - Os elementos são acessados por índices (geralmente começando em 0).
 - **O acesso é rápido**: buscar ou alterar um elemento pelo índice é operação de tempo constante **$O(1)$** .
 - O tamanho costuma ser fixo: ao criar um array, você define quantos elementos ele terá (embora em algumas linguagens modernas, isso seja flexível).
 - Todos os elementos ocupam o mesmo espaço de memória (porque têm o mesmo tipo).

O QUE SÃO ARRAYS?



ARRAYS

*Exemplo de Implementação em JavaScript,
sem o uso de funções nativas*

IMPLEMENTAÇÃO DE UM ARRAY

- Essa implementação manual simula o comportamento de um array básico **sem recorrer** às funções nativas do JavaScript, como **push()**, **pop()**, ou **length**, e usa a lógica de manipulação de índices para armazenar e acessar os dados.
 - Nas futuras implementações de estrutura de dados poderemos usar as funções nativas.
 - Nosso objetivo em estudar sem o uso delas neste momento é entender seu funcionamento.



IMPLEMENTAÇÃO DE UM ARRAY

- Estrutura interna (**items**): Usamos um objeto simples para armazenar os itens do array, onde as chaves são os índices do array.
 - **adicionar(elemento)**: Adiciona um novo elemento ao final do array, usando o índice correspondente ao valor do tamanho atual (**this.tamanho**). Depois, incrementamos o tamanho.
 - **remover()**: Remove o último elemento do array, usando o índice que corresponde a tamanho - 1, e decrementa o tamanho.
 - **obterElemento(indice)**: Acessa e retorna o elemento no índice especificado, se ele for válido (entre 0 e tamanho - 1).
 - **tamanhoArray()**: Retorna o número de elementos no array.
 - **limpar()**: Remove todos os elementos do array, redefinindo o objeto **items** e o tamanho para zero.

IMPLEMENTAÇÃO DO PROTÓTIPO DA CLASSE

```
1  export default class MeuArray {  
2  
3      constructor() { }  
4  
5      adicionar(elemento) { }  
6  
7      remover() { }  
8  
9      obterElemento(indice) { }  
10  
11     tamanhoArray() { }  
12  
13     limpar() { }  
14  
15     editar(indice, novoValor) { }  
16  
17     obterIndice(valor) { }  
18 }
```



IMPLEMENTAÇÃO DO MÉTODO CONSTRUTOR

```
Aula 03 - Array - MeuArray.js

1  export default class MeuArray {
2
3      // Atributo privado, usamos uma lista
4      // para armazenar os itens do array
5      #items = [];
6
7      // Mantemos o controle do
8      // tamanho do array (qnt de itens)
9      #tamanho = 0;
10
11     // Chamando quando um objeto é criado
12     constructor()
13     {
14         console.log("MeuArray criado!");
15     }
```



IMPLEMENTAÇÃO DO MÉTODO adicionar(elemento)

```
● ● ● Aula 03 - Array - MeuArray.js  
17 // Adiciona um elemento ao final do array  
18 adicionar(elemento)  
19 {  
20     // Insere o elemento na posição  
21     // atual do tamanho  
22     this.#items[this.#tamanho] = elemento;  
23  
24     // Incrementa o tamanho  
25     this.#tamanho++;  
26 }
```



IMPLEMENTAÇÃO DO MÉTODO `remove()`

```
Aula 03 - Array - MeuArray.js

28 // Remove o último elemento do array
29 remove()
30 {
31     // Se o array estiver vazio, não há o que remover
32     if (this.#tamanho === 0) {
33         return undefined;
34     }
35
36     // Armazena o último item
37     const ultimoItem = this.#items[this.#tamanho - 1];
38
39     // Remove o último item do array
40     delete this.#items[this.#tamanho - 1];
41
42     // Decrementa o tamanho
43     this.#tamanho--;
44
45     return ultimoItem; // Retorna o item removido
46 }
```



IMPLEMENTAÇÃO DO MÉTODO obterElemento(indice)

```
Aula 03 - Array - MeuArray.js

48 // Acessa o elemento em um índice específico
49 obterElemento(indice)
50 {
51     if (indice < 0 || indice >= this.#tamanho) {
52
53         // Se o índice estiver fora do alcance,
54         // retorna undefined
55         return undefined;
56     }
57
58     // Retorna o item no índice solicitado
59     return this.#items[indice];
60 }
```



IMPLEMENTAÇÃO DO MÉTODO `limpar()`

```
● ● ● Aula 03 - Array - MeuArray.js  
62 // Remove todos os elementos do array  
63 limpar()  
64 {  
65     // Limpa o vetor  
66     this.#items = [];  
67  
68     // Reinicializa o tamanho  
69     this.#tamanho = 0;  
70 }
```



IMPLEMENTAÇÃO DOS MÉTODOS tamanhoArray() E verItens()



Aula 03 - Array - MeuArray.js

```
72 // Retorna o tamanho do array
73 tamanhoArray = () => this.#tamanho;
74
75 // Retorna todos os itens do array
76 verItens = () => this.#items;
```



COMO USAR A CLASSE



Aula 03 - Array - app.js

```
1  import MeuArray from "./MeuArray.js";
2
3  const minha_variavel = new MeuArray();
4
5  minha_variavel.adicionar(10);
6  minha_variavel.adicionar(20);
7  minha_variavel.adicionar(30);
8  console.table(minha_variavel.verItens());
9
10 console.log(minha_variavel.obterElemento(1)); // Saída: 20
11 console.log(minha_variavel.tamanhoArray()); // Saída: 3
12
13 // Saída: 30 (Remove o último elemento)
14 console.log(minha_variavel.remover());
15 console.log(minha_variavel.tamanhoArray()); // Saída: 2
16 console.table(minha_variavel.verItens());
```



EXERCÍCIOS

*Use a classe implementada acima para
resolver os exercícios*

EXERCÍCIO 1

- Uma empresa deseja criar um sistema simples para gerenciar as tarefas da equipe. Cada tarefa será armazenada em um array utilizando a classe **MeuArray**.
- **Requisitos:**
 - Criar uma instância da classe **MeuArray**.
 - Adicionar cinco tarefas ao array.
 - Remover a última tarefa adicionada.
 - Exibir todas as tarefas armazenadas.
- **Perguntas:**
 - O que acontece quando tentamos acessar um índice fora do tamanho do array?
 - Como garantir que a ordem das tarefas seja mantida ao adicionar e remover itens?

EXERCÍCIO 2

- O setor de Recursos Humanos de uma empresa deseja armazenar os nomes dos funcionários que participaram de um treinamento.
- **Requisitos:**
 - Criar uma instância da classe **MeuArray**.
 - Adicionar os nomes de quatro funcionários ao array.
 - Obter o nome do terceiro funcionário que participou.
 - Limpar todos os registros do array.
- **Perguntas:**
 - O que acontece se tentarmos acessar um índice inexistente após limpar o array?
 - Como modificar a classe para garantir que os nomes armazenados sejam únicos?

EXERCÍCIO 3

- O Adicione mais funcionalidades à classe **MeuArray** conforme na imagem ao lado:
 - Implemente dois novos métodos: para editar e para obter o índice de onde um dado valor está.

```
1  class MeuArray {
2      constructor() { }
3      adicionar(elemento) { }
4      remover() { }
5      obterElemento(indice) { }
6      tamanhoArray() { }
7      limpar() { }
8      editar(indice, novoValor) { }
9      obterIndice(valor) { }
10 };
11
12 module.exports = MeuArray;
```

OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: Google Class Room
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva@fatecjahu.edu.br

